COVID-19 Case Study Analysis

NAME : THANNIRU DHARMA NITHIN

REGISTERED E-MAIL :thannirudharmanithin2003@gmail.com

Python ClassRoom link : https://classroom.codingninjas.com/app/domain/data/34918/learn/module/194

Course

Data Analytics – Coding NinjaS

Dataset

covid_19_dataset.xlsx

Description

This case study analyzes global COVID-19 data including confirmed cases, deaths, and recoveries. The goal is to perform data cleaning, transformation, visualization, and statistical analysis to understand the spread and impact of the pandemic.

Introduction

The COVID-19 pandemic, caused by the SARS-CoV-2 virus, emerged in late 2019 and rapidly spread across the globe. Governments and healthcare organizations relied heavily on data analysis to monitor the spread of the virus and implement control measures.

This case study uses datasets containing global COVID-19 confirmed cases, deaths, and recoveries. The analysis will be performed using Python libraries such as Pandas for data manipulation, NumPy for numerical computation, and Matplotlib for visualization.

The study aims to provide insights into the progression of the pandemic across countries and time.

Objective of the Case Study

The objectives of this case study are:

• Demonstrate practical use of Python for data analysis
• Perform data cleaning and handling missing values
• Analyze COVID-19 confirmed cases, deaths, and recoveries
• Generate visualizations to understand trends
• Perform comparative country analysis
• Merge datasets for comprehensive insights

Import Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Task 1 – Data Loading

How do you load the COVID-19 datasets for confirmed cases, deaths, and recoveries into Python using Pandas?

The dataset contains three sheets:

• Confirmed Cases
• Deaths
• Recovered

Each sheet contains cumulative counts of COVID-19 cases for multiple countries over time.

```
file = "covid_19_dataset.xlsx"

confirmed = pd.read_excel(file, sheet_name="covid_19_confirmed_v1")
deaths = pd.read_excel(file, sheet_name="covid_19_deaths_v1")
recovered = pd.read_excel(file, sheet_name="covid_19_recovered_v1")

confirmed.head()
```

```
  Province/State Country/Region       Lat       Long  1/22/20  1/23/20
\
0            NaN    Afghanistan  33.93911  67.709953        0        0

1            NaN        Albania  41.15330  20.168300        0        0

2            NaN        Algeria  28.03390   1.659600        0        0

3            NaN        Andorra  42.50630   1.521800        0        0

4            NaN         Angola -11.20270  17.873900        0        0
```

```
   1/24/20  1/25/20  1/26/20  1/27/20  ...  5/20/21  5/21/21  5/22/21
\
0        0        0        0        0  ...    64575    65080    65486

1        0        0        0        0  ...   132118   132153   132176

2        0        0        0        0  ...   126156   126434   126651

3        0        0        0        0  ...    13569    13569    13569

4        0        0        0        0  ...    31661    31909    32149


   5/23/21  5/24/21  5/25/21  5/26/21  5/27/21  5/28/21  5/29/21
0    65728    66275    66903    67743    68366    69130    70111
1   132209   132215   132229   132244   132264   132285   132297
2   126860   127107   127361   127646   127926   128198   128456
3    13569    13569    13664    13671    13682    13693    13693
4    32441    32623    32933    33338    33607    33944    34180

[5 rows x 498 columns]
```

```
deaths.head()
```

```
  Province/State Country/Region       Lat       Long  1/22/20  1/23/20
\
0            NaN    Afghanistan  33.93911  67.709953        0        0

1            NaN        Albania  41.15330  20.168300        0        0

2            NaN        Algeria  28.03390   1.659600        0        0

3            NaN        Andorra  42.50630   1.521800        0        0

4            NaN         Angola -11.20270  17.873900        0        0


   1/24/20  1/25/20  1/26/20  1/27/20  ...  5/20/21  5/21/21  5/22/21
\
0        0        0        0        0  ...     2772     2782     2792

1        0        0        0        0  ...     2440     2441     2442

2        0        0        0        0  ...     3401     3405     3411

3        0        0        0        0  ...      127      127      127

4        0        0        0        0  ...      704      709      715


   5/23/21  5/24/21  5/25/21  5/26/21  5/27/21  5/28/21  5/29/21
```

```
0     2802     2812     2836     2855     2869     2881     2899
1     2444     2445     2447     2447     2447     2448     2449
2     3418     3426     3433     3440     3448     3455     3460
3      127      127      127      127      127      127      127
4      725      731      735      742      745      749      757

[5 rows x 498 columns]
```

```
recovered.head()
```

```
  Province/State Country/Region       Lat       Long  1/22/20  1/23/20
\
0            NaN    Afghanistan  33.93911  67.709953        0        0

1            NaN        Albania  41.15330  20.168300        0        0

2            NaN        Algeria  28.03390   1.659600        0        0

3            NaN        Andorra  42.50630   1.521800        0        0

4            NaN         Angola -11.20270  17.873900        0        0


   1/24/20  1/25/20  1/26/20  1/27/20  ...  5/20/21  5/21/21  5/22/21
\
0        0        0        0        0  ...    55687    55790    55889

1        0        0        0        0  ...   127869   128425   128601

2        0        0        0        0  ...    87902    88066    88208

3        0        0        0        0  ...    13234    13234    13234

4        0        0        0        0  ...    26483    26513    26775


   5/23/21  5/24/21  5/25/21  5/26/21  5/27/21  5/28/21  5/29/21
0    56035    56295    56518    56711    56962    57119    57281
1   128732   128826   128907   128978   129042   129097   129215
2    88346    88497    88672    88861    89040    89232    89419
3    13234    13234    13263    13381    13405    13416    13416
4    26778    27087    27204    27467    27529    27577    27646

[5 rows x 498 columns]
```

The datasets have been successfully loaded into Pandas DataFrames.
Each dataset contains geographical information along with daily cumulative COVID-19 statistics.

TASK 2 — Data Exploration

Task 2.1 – Structure of Dataset

After loading the datasets, what is the structure of each dataset in terms of rows, columns, and data types?

```
confirmed.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 276 entries, 0 to 275
Columns: 498 entries, Province/State to 5/29/21
dtypes: float64(2), int64(494), object(2)
memory usage: 1.0+ MB
```

```
deaths.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 276 entries, 0 to 275
Columns: 498 entries, Province/State to 5/29/21
dtypes: float64(3), int64(493), object(2)
memory usage: 1.0+ MB
```

```
recovered.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 261 entries, 0 to 260
Columns: 498 entries, Province/State to 5/29/21
dtypes: float64(3), int64(493), object(2)
memory usage: 1015.6+ KB
```

```
print("Confirmed shape:", confirmed.shape)
print("Deaths shape:", deaths.shape)
print("Recovered shape:", recovered.shape)
```

```
Confirmed shape: (276, 498)
Deaths shape: (276, 498)
Recovered shape: (261, 498)
```

The datasets contain hundreds of geographic entries and daily cumulative counts from January 22, 2020 to May 29, 2021.

Task 2.2 – Generate plots of confirmed cases over time for top countries

```
country_confirmed = confirmed.groupby("Country/Region").sum()

country_confirmed = country_confirmed.drop(columns=["Lat", "Long"])
```

```
latest_date = country_confirmed.columns[-1]
top_countries =
country_confirmed[latest_date].sort_values(ascending=False).head(5)

top_countries
```

```
Country/Region
US        33251939
India     27894800
Brazil    16471600
France     5719877
Turkey     5235978
Name: 5/29/21, dtype: int64
```

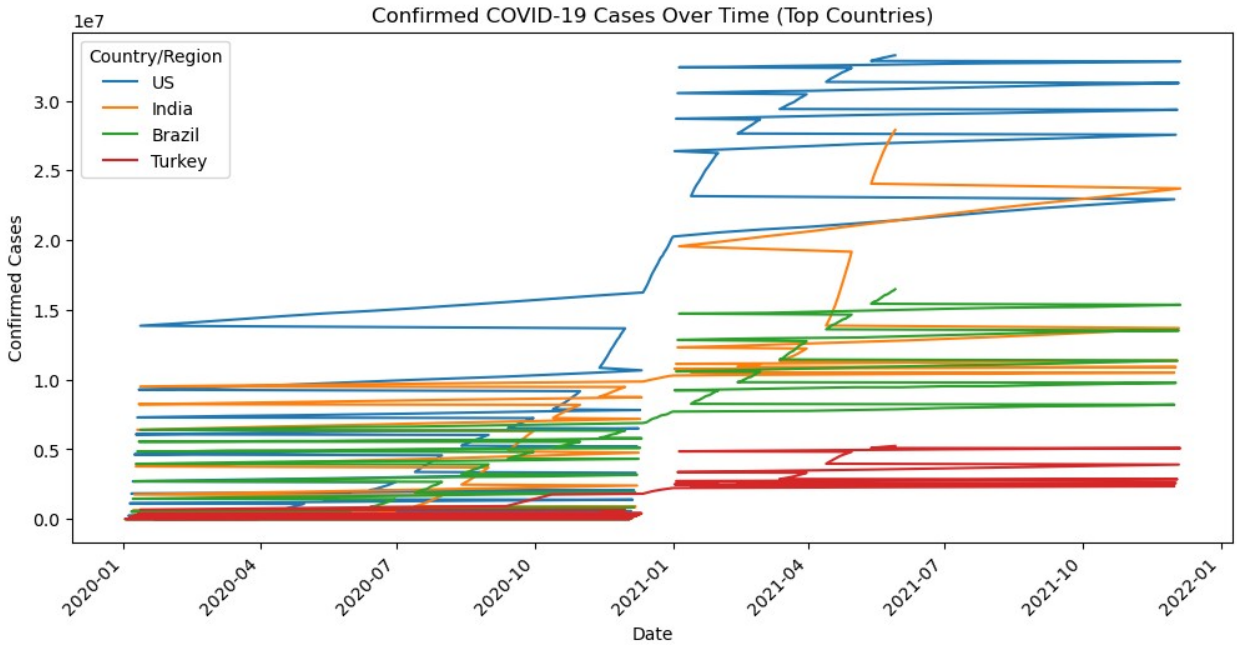
```
top_country_names = top_countries.index

# Extract their data
top_data = country_confirmed.loc[top_countries.index]

# Transpose so dates become rows
top_data = top_data.T
```

Line Plot – Confirmed Cases Over Time

A line plot is the most suitable visualization for time-series data because it shows how values change over time.

```
top_data.index = pd.to_datetime(top_data.index, errors='coerce')
top_data.plot(figsize=(12,6))


plt.title("Confirmed COVID-19 Cases Over Time (Top Countries)")
plt.xlabel("Date")
plt.ylabel("Confirmed Cases")
plt.xticks(rotation=45)

plt.show()
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_2872\885861886.py:1:
UserWarning: Could not infer format, so each element will be parsed
individually, falling back to `dateutil`. To ensure parsing is
consistent and as-expected, please specify a format.
  top_data.index = pd.to_datetime(top_data.index, errors='coerce')
```

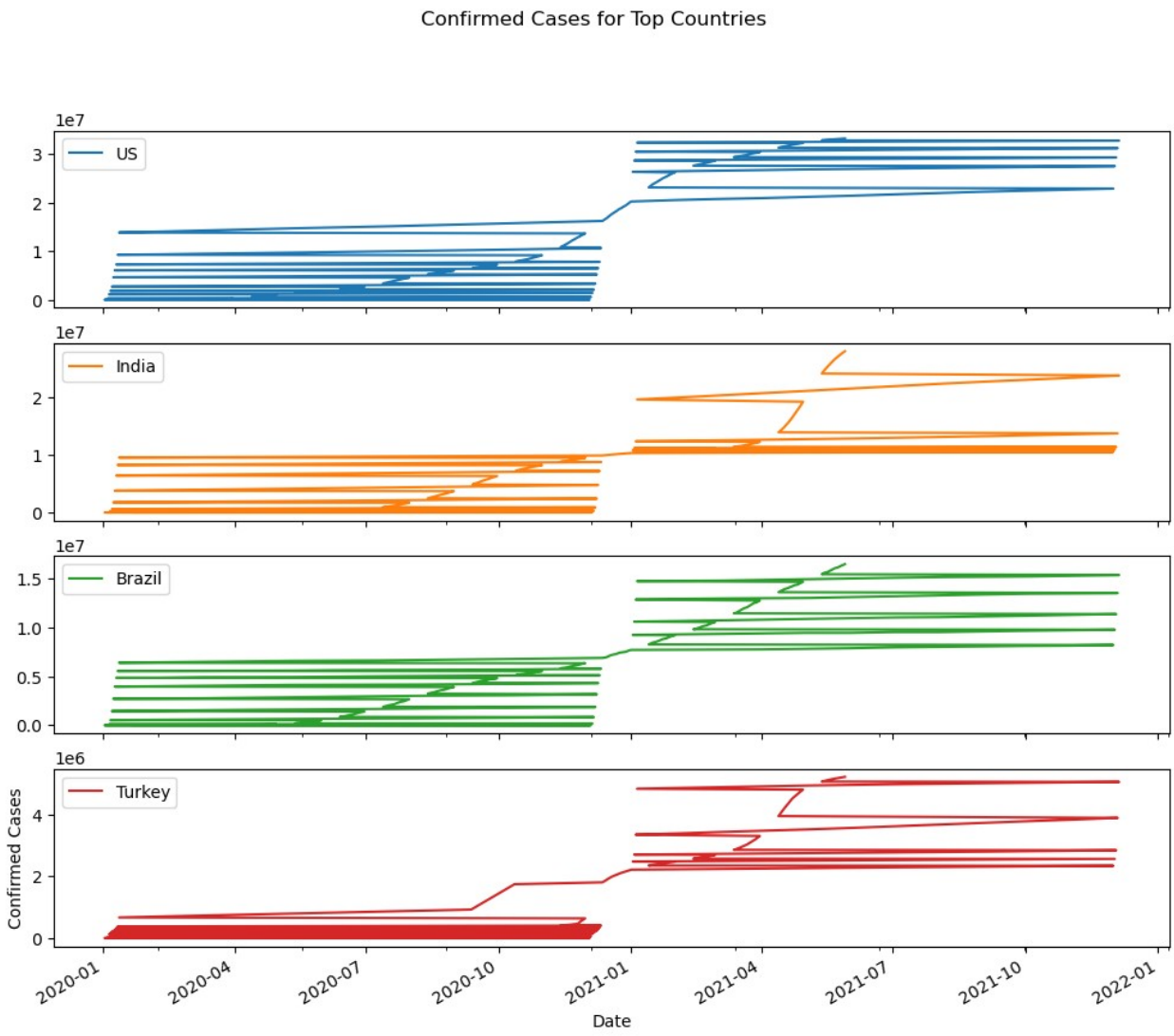


The visualization shows the progression of confirmed cases over time for the countries with the highest infection counts.

Individual Country Trends

To better understand each country's case trend, we can plot each country separately using subplots.

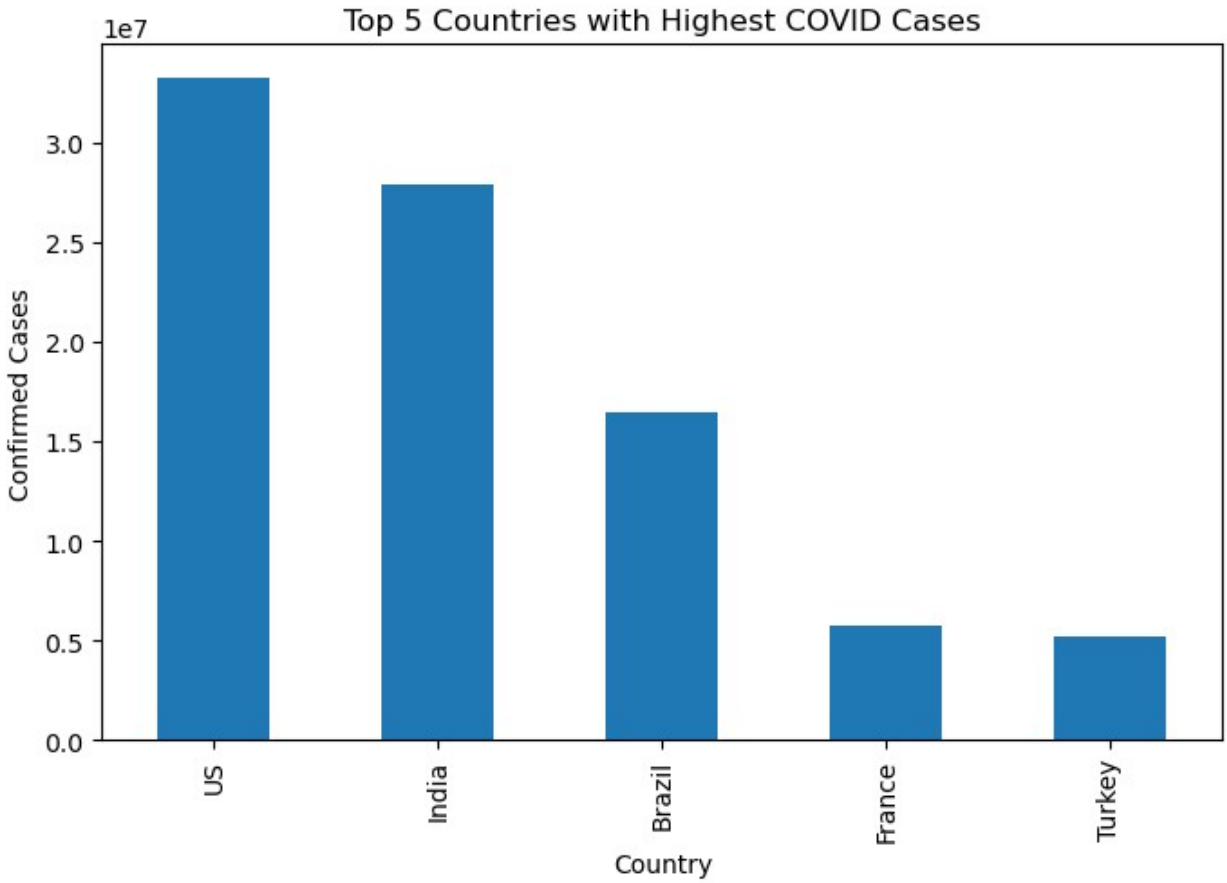
```
top_data.plot(subplots=True, figsize=(12,10), title="Confirmed Cases
for Top Countries")

plt.xlabel("Date")
plt.ylabel("Confirmed Cases")

plt.show()
```



Bar Chart – Latest Confirmed Cases

A bar chart helps compare the total confirmed cases across countries.

```
plt.figure(figsize=(8,5))

top_countries.plot(kind="bar")

plt.title("Top 5 Countries with Highest COVID Cases")
plt.xlabel("Country")
plt.ylabel("Confirmed Cases")

plt.show()
```



Insight

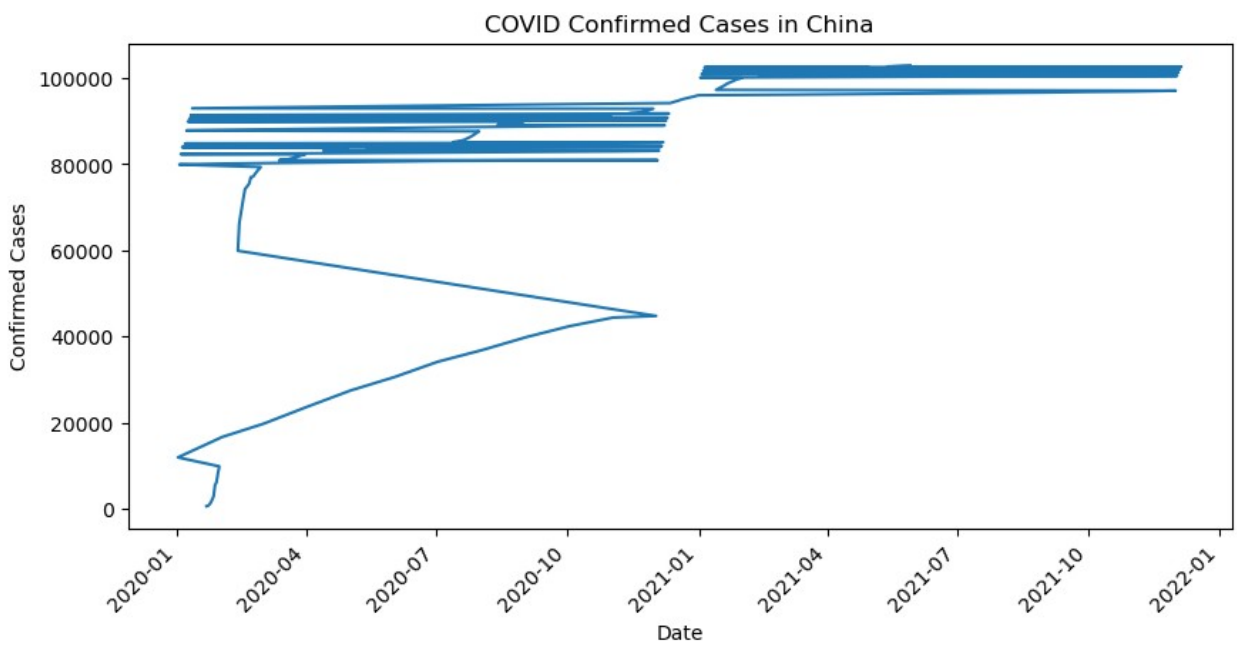
From the visualizations, we can observe how the number of confirmed COVID-19 cases increased rapidly over time in the most affected countries. The line plots highlight the growth trend, while the bar chart shows the relative magnitude of cases among these countries.

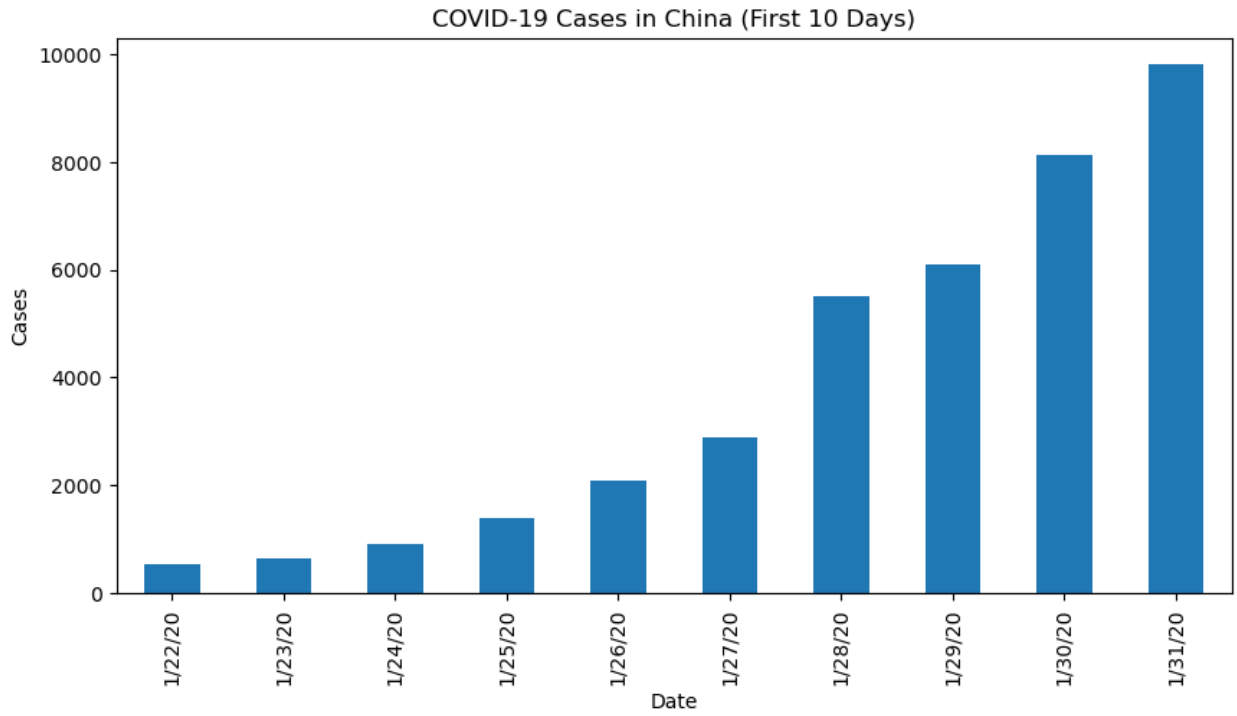
Task 2.3 – Generate plots of confirmed cases over time for China

```
# Filter China data
china_data = confirmed[confirmed["Country/Region"] == "China"]

# Drop non-date columns first
china_data =
china_data.drop(columns=["Province/State","Country/Region","Lat","Long
"])

# Sum all provinces
china_cases = china_data.sum()

# Convert index to datetime
china_cases.index = pd.to_datetime(china_cases.index)
```

```
china_cases.plot(figsize=(10,5))
plt.title("COVID Confirmed Cases in China")
plt.xlabel("Date")
plt.ylabel("Confirmed Cases")
plt.xticks(rotation=45)
plt.show()


china_total.head(10).plot(kind="bar", figsize=(10,5))

plt.title("COVID-19 Cases in China (First 10 Days)")
plt.xlabel("Date")
plt.ylabel("Cases")

plt.show()
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_2872\2888834277.py:11:
UserWarning: Could not infer format, so each element will be parsed
individually, falling back to `dateutil`. To ensure parsing is
consistent and as-expected, please specify a format.
  china_cases.index = pd.to_datetime(china_cases.index)
```





Task 3 – Handling Missing Data

Identify missing values and replace them using forward filling suitable for time-series data.

```
confirmed = confirmed.ffill()
deaths = deaths.ffill()
recovered = recovered.ffill()
```

Forward filling propagates the previous valid observation forward to fill missing values.

Task 4 – Data Cleaning and Preparation

Replace blank values in Province/State column with “All Provinces”.

```
confirmed["Province/State"] = confirmed["Province/State"].fillna("All
Provinces")
deaths["Province/State"] = deaths["Province/State"].fillna("All
Provinces")
recovered["Province/State"] = recovered["Province/State"].fillna("All
Provinces")
```

Task 5 Independent Dataset Analysis

Task 5.1 – Analyze the peak number of daily new cases in Germany, France, and Italy.

```
countries = ["Germany","France","Italy"]

country_data = confirmed[confirmed["Country/Region"].isin(countries)]

country_sum =
country_data.groupby("Country/Region").sum().drop(columns=["Lat","Long
"], errors="ignore")

# keep only numeric columns (dates)
country_sum = country_sum.select_dtypes(include="number")

daily_cases = country_sum.diff(axis=1)

peak_cases = daily_cases.max(axis=1)

peak_cases
```

```
Country/Region
France     117900.0
Germany     49044.0
Italy       40902.0
dtype: float64
```

Task 5.2 – Compare recovery rates between Canada and Australia as of Dec 31 2020

```
date = "12/31/20"

confirmed_country = confirmed.groupby("Country/Region").sum()
recovered_country = recovered.groupby("Country/Region").sum()

recovery_rate = recovered_country[date] / confirmed_country[date]

recovery_rate.loc[["Canada","Australia"]]
```

```
Country/Region
Canada       0.844679
Australia    0.793843
Name: 12/31/20, dtype: float64
```

Task 5.3 – Distribution of death rates among provinces in Canada

```
canada_confirmed = confirmed[confirmed["Country/Region"]=="Canada"]
canada_deaths = deaths[deaths["Country/Region"]=="Canada"]
```

```
latest_date = confirmed.columns[-1]

death_rate = canada_deaths[latest_date] /
canada_confirmed[latest_date]

result = pd.DataFrame({
"Province": canada_confirmed["Province/State"],
"Death Rate": death_rate
})

result
```

```
                     Province  Death Rate
39                    Alberta    0.009760
40           British Columbia    0.011784
41           Diamond Princess         inf
42             Grand Princess    0.000000
43                   Manitoba    0.020693
44              New Brunswick    0.019626
45  Newfoundland and Labrador    0.004608
46      Northwest Territories    0.000000
47                Nova Scotia    0.015190
48                    Nunavut    0.006173
49                    Ontario    0.016193
50       Prince Edward Island    0.000000
51                     Quebec    0.030090
52     Repatriated Travellers    0.000000
53               Saskatchewan    0.011557
54                      Yukon    0.023810
```

Task 6 Data Transformation

Task 6.1 – Transform deaths dataset from wide format to long format

```
deaths_long = deaths.melt(
    id_vars=["Province/State","Country/Region","Lat","Long"],
    var_name="Date",
    value_name="Deaths"
)

deaths_long["Date"] = pd.to_datetime(deaths_long["Date"],
format="%m/%d/%y")
```

The melt() function converts the dataset from wide format to long format. Each date column becomes a row with two new columns: Date and Deaths. This makes the data easier to analyze and visualize over time.

Task 6.2 – Total deaths reported per country

```
total_deaths_country = deaths.groupby("Country/Region").sum().iloc[:,-1]

total_deaths_country.sort_values(ascending=False).head(10)
```

```
Country/Region
US                 594306
Brazil             461057
India              325972
Mexico             223455
United Kingdom     128037
Italy              126002
Russia             118781
France             109518
Germany             88413
Colombia            87747
Name: 5/29/21, dtype: int64
```

Task 6.3 – Top 5 countries with highest average daily deaths

```
# Group by country
deaths_country = deaths.groupby("Country/Region").sum()

# Keep only numeric columns (date columns)
deaths_country = deaths_country.select_dtypes(include="number")

# Calculate daily deaths
daily_deaths = deaths_country.diff(axis=1)

# Average daily deaths
avg_daily_deaths = daily_deaths.mean(axis=1)

# Top 5 countries
avg_daily_deaths.sort_values(ascending=False).head(5)
```

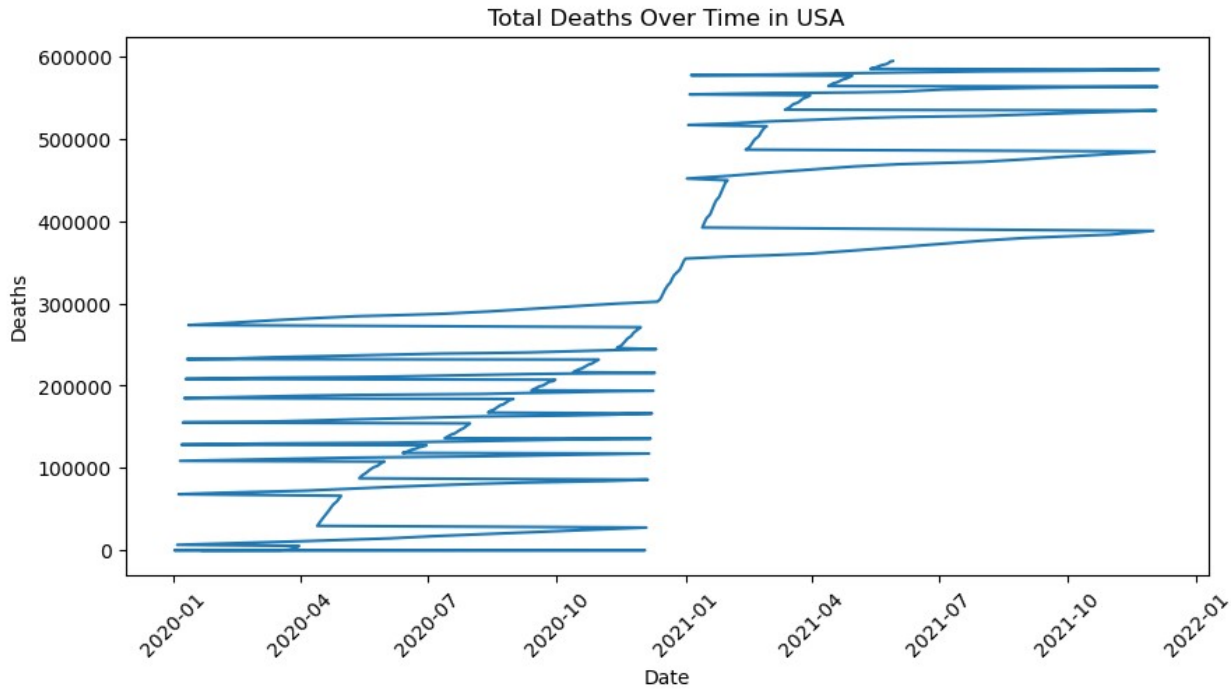
```
Country/Region
US                 1200.537374
Brazil              931.457040
India               658.487690
Mexico              451.376496
United Kingdom      258.131145
dtype: float64
```

The data was grouped by country and the total deaths were calculated. The difference between consecutive days was computed to estimate daily deaths. Then the average daily deaths for each country were calculated and the top 5 countries with the highest averages were identified.

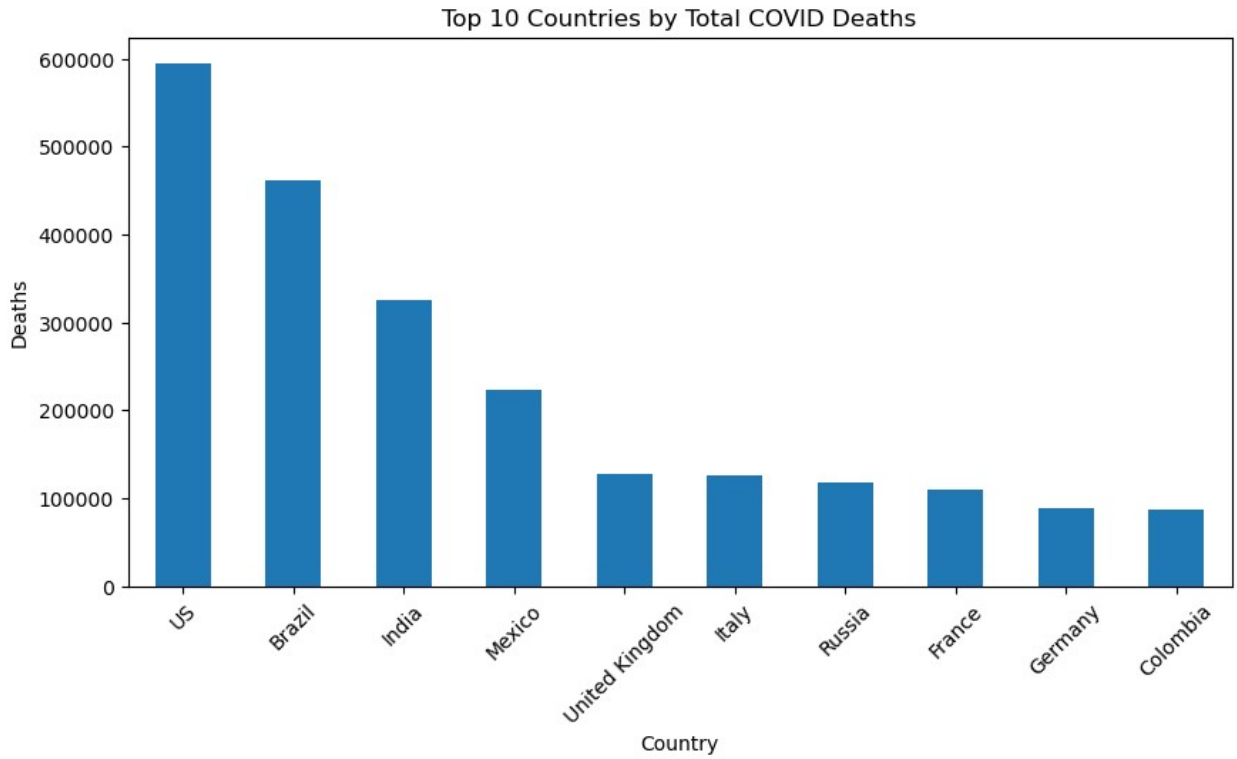
Task 6.4 – Evolution of deaths in United States

```
usa_deaths = deaths[deaths["Country/Region"] == "US"]

# remove non-date columns first
usa_deaths =
usa_deaths.drop(columns=["Province/State","Country/Region","Lat","Long
"], errors="ignore")

# sum all provinces
usa_total = usa_deaths.sum()

# convert index to datetime
usa_total.index = pd.to_datetime(usa_total.index)

# plot
plt.figure(figsize=(10,5))
plt.plot(usa_total.index, usa_total.values)

plt.title("Total Deaths Over Time in USA")
plt.xlabel("Date")
plt.ylabel("Deaths")
plt.xticks(rotation=45)

plt.show()
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_2872\7687440.py:10:
UserWarning: Could not infer format, so each element will be parsed
individually, falling back to `dateutil`. To ensure parsing is
consistent and as-expected, please specify a format.
  usa_total.index = pd.to_datetime(usa_total.index)
```




```
deaths_country = deaths.groupby("Country/Region").sum()

deaths_country = deaths_country.drop(columns=["Lat","Long"],
errors="ignore")

latest_deaths = deaths_country.iloc[:,-1]

top10 = latest_deaths.sort_values(ascending=False).head(10)

plt.figure(figsize=(10,5))
top10.plot(kind="bar")

plt.title("Top 10 Countries by Total COVID Deaths")
plt.xlabel("Country")
plt.ylabel("Deaths")

plt.xticks(rotation=45)
plt.show()
```



Task 7 Data Merging

Task 7.1 – Merge confirmed, deaths, and recovered datasets

```
confirmed_long["Date"] = pd.to_datetime(confirmed_long["Date"],
format="%m/%d/%y")
deaths_long["Date"] = pd.to_datetime(deaths_long["Date"],
format="%m/%d/%y")
recovered_long["Date"] = pd.to_datetime(recovered_long["Date"],
format="%m/%d/%y")

merged = confirmed_long.merge(
    deaths_long,
    on=["Province/State","Country/Region","Lat","Long","Date"]
)

merged = merged.merge(
    recovered_long,
    on=["Province/State","Country/Region","Lat","Long","Date"]
)
```

```
merged.head()
```

```
  Province/State Country/Region       Lat       Long       Date
Confirmed  \
0  All Provinces    Afghanistan  33.93911  67.709953 2020-01-22
0
1  All Provinces        Albania  41.15330  20.168300 2020-01-22
0
2  All Provinces        Algeria  28.03390   1.659600 2020-01-22
0
3  All Provinces        Andorra  42.50630   1.521800 2020-01-22
0
4  All Provinces         Angola -11.20270  17.873900 2020-01-22
0

   Deaths  Recovered
0     0.0        0.0
1     0.0        0.0
2     0.0        0.0
3     0.0        0.0
4     0.0        0.0
```

Task 7.2 – Monthly sum of confirmed cases, deaths and recoveries

```
merged["Date"] = pd.to_datetime(merged["Date"])

merged["Month"] = merged["Date"].dt.to_period("M")

monthly_data = merged.groupby(["Country/Region","Month"])
[["Confirmed","Deaths","Recovered"]].sum()

monthly_data.head()
```

```
                         Confirmed   Deaths  Recovered
Country/Region Month
Afghanistan    2020-01     252263   8630.0   176248.0
               2020-02     253826   8700.0   177059.0
               2020-03     256554   8796.0   178750.0
               2020-04     279293   9526.0   183696.0
               2020-05     442303  12675.0   200799.0
```

Task 7.3 – Monthly analysis for USA, Italy and Brazil

```
selected = monthly_data.loc[["US","Italy","Brazil"]]

selected
```

```
                          Confirmed     Deaths   Recovered
Country/Region Month
US             2020-01   46953196  1365722.0  16743079.0
               2020-02   47520544  1376429.0  17001228.0
               2020-03   49251512  1413331.0  17266040.0
               2020-04   63906892  2270586.0  19024548.0
               2020-05   79852288  3284433.0  24126537.0
...                           ...        ...         ...
Brazil         2021-08   57015749  1465733.0  50532708.0
               2021-09   57355956  1475579.0  50840764.0
               2021-10   57622258  1483169.0  50990323.0
               2021-11   57887966  1491347.0  51365364.0
               2021-12   58201677  1499935.0  51532281.0

[72 rows x 3 columns]
```

Task 8 Combined Data Analysis

Task 8.1 – Countries with highest average death rates in 2020

```
merged["Death_Rate"] = merged["Deaths"] / merged["Confirmed"]

merged_2020 = merged[merged["Date"].dt.year == 2020]

avg_death_rate = merged_2020.groupby("Country/Region")
["Death_Rate"].mean()

avg_death_rate.sort_values(ascending=False).head(3)
```

```
Country/Region
Sudan               inf
Yemen          0.249532
MS Zaandam     0.219036
Name: Death_Rate, dtype: float64
```

Task 8.2 – Compare recoveries and deaths in South Africa

```
sa = merged[merged["Country/Region"]=="South Africa"]

sa[["Recovered","Deaths"]].sum()
```

```
Recovered    310037573.0
Deaths        10250036.0
dtype: float64
```

Task 8.3 – Recovery ratio for USA monthly (Mar 2020 – May 2021)

```
usa = merged[merged["Country/Region"]=="US"]

usa_monthly = usa.groupby("Month")[["Confirmed","Recovered"]].sum()

usa_monthly["Recovery_Ratio"] = usa_monthly["Recovered"] /
usa_monthly["Confirmed"]

usa_monthly.loc["2020-03":"2021-05"]
```

	Confirmed	Recovered	Recovery_Ratio
Month			
2020-03	49251512	17266040.0	0.350569
2020-04	63906892	19024548.0	0.297692
2020-05	79852288	24126537.0	0.302140
2020-06	91706926	29321196.0	0.319727
2020-07	126004651	41339627.0	0.328080
2020-08	158562744	56051131.0	0.353495
2020-09	175576873	66181707.0	0.376939
2020-10	213270323	82616511.0	0.387379
2020-11	273831307	101592133.0	0.371003
2020-12	401432296	25708091.0	0.064041
2021-01	609930951	0.0	0.000000
2021-02	589142237	0.0	0.000000
2021-03	707605918	0.0	0.000000
2021-04	713557698	0.0	0.000000
2021-05	702550516	0.0	0.000000

Conclusion

This case study analyzed COVID-19 datasets to understand the progression of the pandemic across countries.

Key operations performed:

- Data loading and exploration
- Handling missing values
- Data cleaning
- Data transformation
- Data merging
- Country-level comparative analysis

The analysis demonstrated how Python can be used to process real-world datasets and generate meaningful insights. Visualization and statistical metrics helped identify trends in confirmed cases, recovery rates, and death rates across different regions.